Project Design Phase-I Solution Architecture

Quotes Recommendation Chatbot using Rasa NLU

Date	07 March 2026
Team ID	SmartBridge-EL-2026
Project Name	Quotes Recommendation Chatbot using Rasa NLU
Maximum Marks	As per evaluation criteria

SmartBridge Experiential Learning Program
Team Members: Vansh Malhotra | Varun Gaikwad | Vedika Tandulwadkar | Vidisha Jain

Solution Architecture:

Solution architecture is a complex process – with many sub-processes – that bridges the gap between business problems and technology solutions. Its goals are to:

- Find the best tech solution to solve the existing business problem of delivering quotes via a conversational chatbot.
- Describe the structure, characteristics, behaviour, and other aspects of the software to project stakeholders.
- Define features, development phases, and solution requirements.
- Provide specifications according to which the solution is defined, managed, and delivered.

Solution Architecture Diagram:

The diagram below shows all components of the Quotes Recommendation Chatbot and how they connect and communicate at runtime:

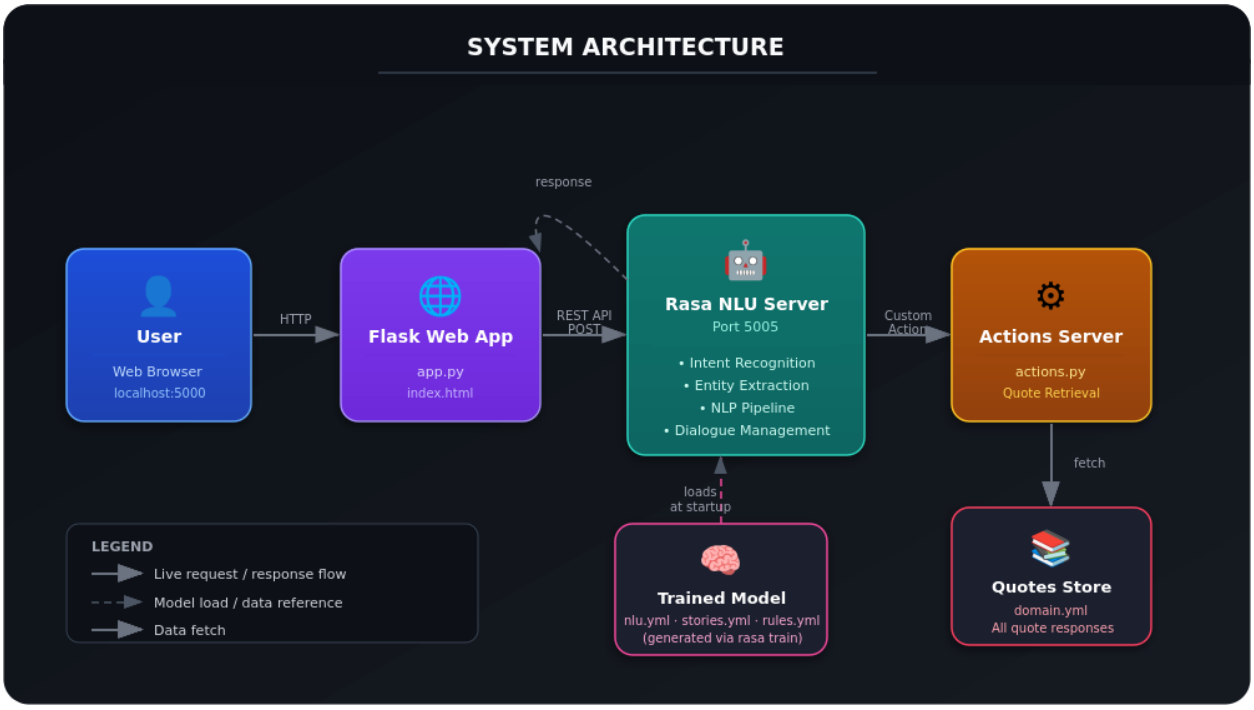

Figure 1: System Architecture — Quotes Recommendation Chatbot using Rasa NLU

Key components of the architecture:

- User (Browser) — the end user who opens the chat UI at localhost:5000 and types messages.
- Flask Web App (port 5000) — serves index.html and forwards messages to the Rasa REST API.
- Rasa NLU Server (port 5005) — classifies intent from user text using the trained ML model.
- Actions Server (actions.py) — triggered by Rasa; selects and returns the appropriate quote.
- Quotes Store (domain.yml) — all predefined quotes stored per category, loaded in memory.
- Trained Model — generated from nlu.yml and stories.yml via 'rasa train'; loaded at startup.

Project Flow Diagram

The diagram below shows the step-by-step flow of a single user interaction from opening the browser to receiving a quote response:

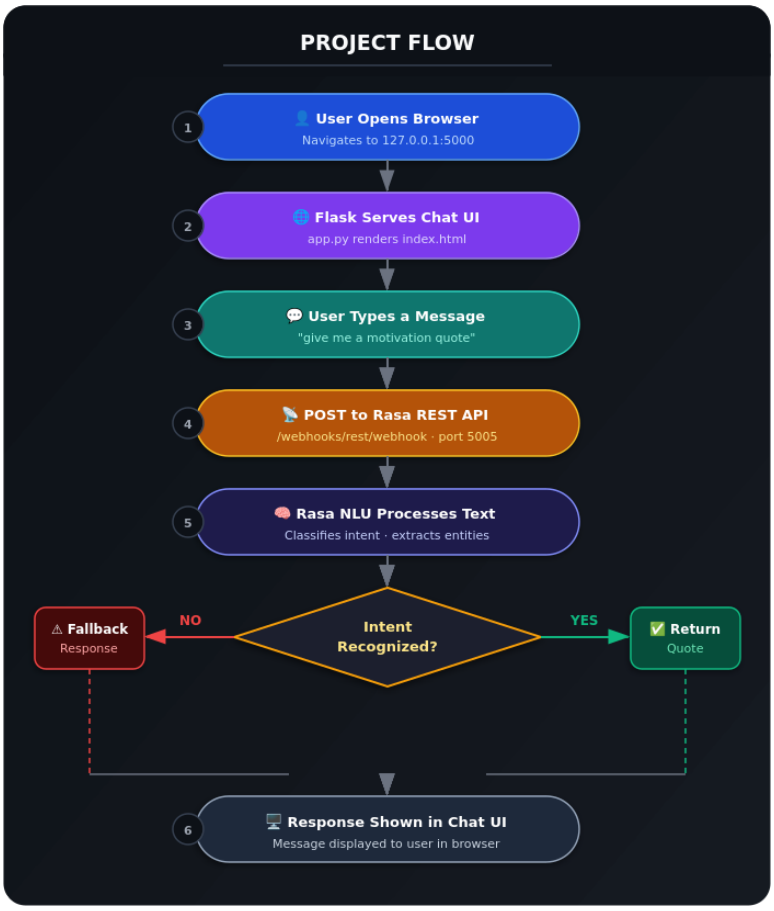

Figure 2: Project Flow — End-to-end user interaction

Table 1: Components & Technologies

S.No	Component	Description	Technology
1	User Interface	Chat UI — how the user interacts with the chatbot	HTML5, CSS3, JavaScript, Jinja2 (Flask templates)
2	Web Application	Serves the chat UI and forwards messages to the Rasa REST API	Python 3.9, Flask 2.x, Requests library
3	NLU Engine	Classifies user intent from natural language text input	Rasa NLU 3.6.x, spaCy, TensorFlow
4	Dialogue Management	Controls conversation flow using rules, stories and policies	Rasa Core — rules.yml, stories.yml
5	Custom Actions	Python functions that execute when an intent is triggered — returns quote	Python 3.9, actions.py, Rasa SDK
6	Quotes Data Store	All predefined quotes stored per category, loaded at runtime	YAML — domain.yml (in-memory)
7	Trained NLP Model	ML model trained on labelled intent data for text classification	Rasa NLU pipeline, spaCy en_core_web_md
8	REST API	Communication interface between Flask web app and Rasa server	Rasa REST Webhook — /webhooks/rest/webhook
9	Infrastructure	All components run locally — no cloud deployment for MVP	Local server, Windows/Linux, Python venv, port 5000/5005

Table 2: Application Characteristics

S.No	Characteristic	Description	Technology
1	Open-Source Frameworks	All technologies are fully open-source with no licensing costs	Rasa (Apache 2.0), Flask (BSD), Python (PSF), spaCy (MIT)
2	Security	No user data stored. CORS enabled for local dev. No authentication required for MVP	CORS (*), local network only, no persistent storage
3	Scalable Architecture	3-tier architecture separates UI, logic, and data. Rasa and Actions run as independent services — each can be scaled independently	Microservice-style: Flask + Rasa server + Actions server
4	Availability	Application is available for the full session. Cloud deployment would enable 24/7 availability	Local: manual start; Future: Docker + AWS/Azure

S.No	Characteristic	Description	Technology
5	Performance	Response time < 2s locally. Rasa processes NLU in-memory using pre-loaded trained model	In-memory model inference, Flask development server

Data / Request Flow

Step	Process	Data Transferred
1	User sends message via browser	Text: 'give me a motivation quote'
2	Flask forwards to Rasa REST API	HTTP POST — JSON: {sender: user, message: text}
3	Rasa NLU classifies intent	Intent label: motivational_quote (with confidence score)
4	Rasa triggers custom action	Action: action_motivational_quote
5	actions.py fetches quote	Quote string selected from domain.yml responses
6	Response returned to Flask	JSON: {text: 'Your only limit is your mind.'}
7	Flask renders reply in browser	Quote displayed as bot message chat bubble in the UI

Deployment Architecture

Parameter	Details
Environment	Local development machine — Windows / Linux
Language Runtime	Python 3.9 virtual environment (venv)
Server 1 (Terminal 1)	Rasa server: rasa run --enable-api --cors "*" → port 5005
Server 2 (Terminal 2)	Flask app: python app.py → port 5000
User Access	Browser at http://127.0.0.1:5000
Local Dependencies	Python 3.9, Rasa 3.6.x, Flask, spaCy, TensorFlow — installed via requirements.txt
Future Deployment	Docker containerisation + Cloud deployment on AWS / Azure

References

GitHub Repository: https://github.com/VarunGaikwad-tech/quotes-recommendation-chatbot-rasa

Rasa Documentation: https://rasa.com/docs/rasa/

Flask Documentation: https://flask.palletsprojects.com/

Architecture Reference: https://c4model.com/